Representation of numbers

Outline

Introduction

Representation of natural numbers

Representation of integer numbers

Representation of real numbers

In summary

Introduction

Algorithms manipulate data

- An algorithm manipulates data: before describing how it works, we have to agree on how its data are **written down**
- For numbers the answer is precise: a computer stores every number as a **finite sequence of symbols**
- With a given number of symbols, only **finitely many** numbers can be written

Two important consequences

- Every representation has a **representation interval**: numbers outside of it cannot be stored
- The representation of real numbers is, in general, only **approximate**

Three representations

In this chapter we study:

- the representation of **natural numbers** in a positional numeral system with base b, in particular the binary system used by computers;
- the representation of **integer numbers**, first in sign-magnitude and then in two's complement, the representation used by modern computers;
- the representation of **real numbers** in floating point, according to the IEEE 754 standard.

For each representation we compute the largest and the smallest number that can be represented with a given number of digits, and the minimum number of digits needed to represent a given number.

Representation of natural numbers

Introduction: Representation of natural numbers

- We define positional numeral systems and their base
- We compute the number represented by a sequence of digits in base b
- We find which natural numbers can be written with k digits, and how many digits a given number needs
- The binary system is the one used by computers

Positional numeral systems

- We are used to writing numbers in the decimal system
- The same digit has a different value depending on where it appears
- In 555 the three digits are equal, but they stand for 500, 50 and 5
- This is the idea of a **positional numeral system**

Definition 1: positional numeral system

*A **positional numeral system** is a way to encode the natural numbers using a **sequence of digits**, where each digit (single symbol) contributes differently depending on its position.*

Definition 2: base of a positional numeral system

*The number of different digits in a positional numeral system is called the **base** b of the system.*

Bases

- The base b of a positional numeral system must be greater than or equal to two
- The **unary system** is not a positional numeral system: natural numbers are represented by repeating the digit 1 as many times as necessary
  - e.g., 3 is represented as 111

The principal positional numeral systems

Positional numeral system	base b	different digits
Binary system	2	0, 1
Octal system	8	0, 1, 2, 3, 4, 5, 6, 7
Decimal system	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Hexadecimal system	16	$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$

- In the hexadecimal system the six letters A, B, C, D, E, F are digits, with the values 10, 11, 12, 13, 14, 15

Definition 3: sequence of digits

*A **sequence of** k **digits** representing a natural number in base b is denoted as:*

$$\left(d_{k-1} \quad d_{k-2} \quad \dots \quad d_2 \quad d_1 \quad d_0\right)_b$$

*where the positions are counted from right to left, starting at 0 and increasing up to $k-1$. The **Least Significant Digit** (LSD) is d_0. The **Most Significant Digit** (MSD) is d_{k-1}.*

The value of a sequence of digits

- The digit in position i is multiplied by the i-th power of the base
- Summing all these contributions we obtain the number represented by the sequence

Given a sequence of k digits in a positional numeral system in base b, the natural number it represents in base ten is given by the following sequence of digits:

$$\left(\sum_{i=0}^{k-1} d_i \; b^i \right)_{10}$$

Example: sequences of digits (1/2)

- The number one hundred written in the four positional numeral systems of the table above
- The value of each sequence is computed with the formula $\sum_{i=0}^{k-1} d_i \, b^i$, position by position, starting from the least significant digit d_0
- We first look at the binary system, then at the other three

Example: sequences of digits (2/2)

Sequence	b	k	decimal representation
$(100)_2$	2	3	$(0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2)_{10} = (4)_{10}$
$(1100100)_2$	2	7	$(0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 + 1 \cdot 2^5 + 1 \cdot 2^6)_{10}$ $= (4 + 32 + 64)_{10} = (100)_{10}$
$(144)_8$	8	3	$(4 \cdot 8^0 + 4 \cdot 8^1 + 1 \cdot 8^2)_{10} = (4 + 32 + 64)_{10} = (100)_{10}$
$(100)_{10}$	10	3	$(0 \cdot 10^0 + 0 \cdot 10^1 + 1 \cdot 10^2)_{10} = (100)_{10}$
$(64)_{16}$	16	2	$(4 \cdot 16^0 + 6 \cdot 16^1)_{10} = (4 + 96)_{10} = (100)_{10}$

The same sequence of digits 100 represents different numbers in different bases, and the same number needs more digits when the base is smaller.

With $k = 4$ binary digits we can write $2^4 = 16$ different sequences.

Let us list all of them.

Example: all the sequences of four binary digits (1/2)

The 16 sequences represent the natural numbers from 0 to 15:

Sequence	b	k	decimal representation
$(0000)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (0)_{10}$
$(0001)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (1)_{10}$
$(0010)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (2)_{10}$
$(0011)_2$	2	4	$(0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (3)_{10}$
$(0100)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (4)_{10}$
$(0101)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (5)_{10}$
$(0110)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (6)_{10}$
$(0111)_2$	2	4	$(0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (7)_{10}$

Example: all the sequences of four binary digits (2/2)

Sequence	b	k	decimal representation
$(1000)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (8)_{10}$
$(1001)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (9)_{10}$
$(1010)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (10)_{10}$
$(1011)_2$	2	4	$(1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (11)_{10}$
$(1100)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0)_{10} = (12)_{10}$
$(1101)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)_{10} = (13)_{10}$
$(1110)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = (14)_{10}$
$(1111)_2$	2	4	$(1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)_{10} = (15)_{10}$

Notation

Typically, for sequences of digits in base ten,
the parentheses are omitted,

and if no parentheses are present,
the base is assumed to be ten.

Representation intervals

- The table of all the sequences of four binary digits stops at 15: with four binary digits there is no way of writing 16
- In general, a fixed number k of digits can only represent the natural numbers up to a certain bound
- We now compute this bound

Observation 1

Given k digits in base b, the ***largest natural number*** *that can be represented is:*

$$b^k - 1$$

and the associated ***representation interval*** *for the natural numbers is:*

$$\left[\ 0,\ b^k - 1\ \right]$$

Proof (1/2)

The largest natural number in base b with k digits consists of only the largest digit $(b-1)$ repeated k times:

$$\left(\; (b-1)_{k-1} \;\; (b-1)_{k-2} \;\; \ldots \;\; (b-1)_1 \;\; (b-1)_0 \; \right)_b$$

Using the sum of the geometric progression (valid since $b \geq 2$)

$$\sum_{i=0}^{k-1} b^i = \frac{b^k - 1}{b-1}$$

the largest natural number in decimal is:

$$\sum_{i=0}^{k-1} (b-1)\, b^i = (b-1) \sum_{i=0}^{k-1} b^i = (b-1)\, \frac{b^k - 1}{b-1} = b^k - 1$$

Proof (2/2)

- The smallest natural number is 0, represented by k digits equal to 0
- Every natural number between 0 and $b^k - 1$ can be represented:
  - the b^k different sequences of k digits represent b^k different numbers
  - all of them are in the interval $[0, b^k - 1]$
  - the interval contains exactly b^k natural numbers

□

Bits and bytes

Digits in the binary system are called **bits**

and 8 bits are called a **byte**.

Example: representation intervals in binary

For $b = 2$ the representation interval with k bits is $[0, 2^k - 1]$. The last rows correspond to 1, 2, 4 and 8 bytes.

k	b	$b^k - 1$	representation interval
1	2	$2^1 - 1 = 1$	$[0, 1]$
2	2	$2^2 - 1 = 3$	$[0, 3]$
3	2	$2^3 - 1 = 7$	$[0, 7]$
4	2	$2^4 - 1 = 15$	$[0, 15]$
8	2	$2^8 - 1 = 255$	$[0, 255]$
16	2	$2^{16} - 1 = 65{,}535$	$[0, 65{,}535]$
32	2	$2^{32} - 1 = 4{,}294{,}967{,}295$	$[0, 4{,}294{,}967{,}295]$[1]
64	2	$2^{64} - 1$	$[0, 2^{64} - 1]$[2]

[1] Representation interval of `unsigned int` in C and C++ (4 bytes).
[2] Representation interval of `unsigned long long int` in C and C++ (8 bytes);
$2^{64} = 18{,}446{,}744{,}073{,}709{,}551{,}616$.

The observation can also be read in the opposite direction:

given a natural number n,
how many digits do we need to write it?

Observation 2

The minimum number of digits to write a natural number $n \geq 1$ *in base* b *is:*

$$\lfloor \log_b n \rfloor + 1$$

Proof

By Observation 1, the number n can be written with k digits if and only if:

$$n \leq b^k - 1 \quad \iff \quad n < b^k \quad \iff \quad \log_b n < k$$

- the first equivalence holds because n and b^k are integers
- the second holds because the logarithm in base $b \geq 2$ is an increasing function

The smallest integer k strictly greater than $\log_b n$ is $\lfloor \log_b n \rfloor + 1$.

Accordingly, $\lfloor \log_b n \rfloor + 1$ is the minimum number of digits to write a natural number n in base b. □

Example: minimum number of digits of one hundred

We compute the minimum number of digits of $n = 100$ in base 2 and in base 10.

- Step 1 (base 2): since $2^6 = 64 \leq 100 < 128 = 2^7$, we have $6 \leq \log_2 100 < 7$ (indeed $\log_2 100 \approx 6.644$), hence:

$$\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$$

- Step 2 (base 10): since $10^2 = 100$, we have $\log_{10} 100 = 2$, hence:

$$\lfloor \log_{10} 100 \rfloor + 1 = 2 + 1 = 3$$

Both results agree with the example on the number one hundred: $(1100100)_2$ has 7 digits and $(100)_{10}$ has 3 digits.

Example: minimum number of digits in binary (1/2)

For the powers of two $n = 2^m$ we have $\log_2 n = m$, so the minimum number of bits is $m + 1$: one digit 1 followed by m digits 0.

n	b	$\log_b n$	$\lfloor \log_b n \rfloor + 1$
$2 = 2^1$	2	$\log_2 2 = 1$	$\lfloor \log_2 2 \rfloor + 1 = 2$
$4 = 2^2$	2	$\log_2 4 = 2$	$\lfloor \log_2 4 \rfloor + 1 = 3$
$8 = 2^3$	2	$\log_2 8 = 3$	$\lfloor \log_2 8 \rfloor + 1 = 4$
$16 = 2^4$	2	$\log_2 16 = 4$	$\lfloor \log_2 16 \rfloor + 1 = 5$
$32 = 2^5$	2	$\log_2 32 = 5$	$\lfloor \log_2 32 \rfloor + 1 = 6$
$64 = 2^6$	2	$\log_2 64 = 6$	$\lfloor \log_2 64 \rfloor + 1 = 7$
$128 = 2^7$	2	$\log_2 128 = 7$	$\lfloor \log_2 128 \rfloor + 1 = 8$
$256 = 2^8$	2	$\log_2 256 = 8$	$\lfloor \log_2 256 \rfloor + 1 = 9$
$512 = 2^9$	2	$\log_2 512 = 9$	$\lfloor \log_2 512 \rfloor + 1 = 10$
$1{,}024 = 2^{10}$	2	$\log_2 1{,}024 = 10$	$\lfloor \log_2 1{,}024 \rfloor + 1 = 11$

Example: minimum number of digits in binary (2/2)

n	b	$\log_b n$	$\lfloor \log_b n \rfloor + 1$
$2{,}048 = 2^{11}$	2	$\log_2 2{,}048 = 11$	$\lfloor \log_2 2{,}048 \rfloor + 1 = 12$
$4{,}096 = 2^{12}$	2	$\log_2 4{,}096 = 12$	$\lfloor \log_2 4{,}096 \rfloor + 1 = 13$
$8{,}192 = 2^{13}$	2	$\log_2 8{,}192 = 13$	$\lfloor \log_2 8{,}192 \rfloor + 1 = 14$
$16{,}384 = 2^{14}$	2	$\log_2 16{,}384 = 14$	$\lfloor \log_2 16{,}384 \rfloor + 1 = 15$
$32{,}768 = 2^{15}$	2	$\log_2 32{,}768 = 15$	$\lfloor \log_2 32{,}768 \rfloor + 1 = 16$
$65{,}536 = 2^{16}$	2	$\log_2 65{,}536 = 16$	$\lfloor \log_2 65{,}536 \rfloor + 1 = 17$
$131{,}072 = 2^{17}$	2	$\log_2 131{,}072 = 17$	$\lfloor \log_2 131{,}072 \rfloor + 1 = 18$
$262{,}144 = 2^{18}$	2	$\log_2 262{,}144 = 18$	$\lfloor \log_2 262{,}144 \rfloor + 1 = 19$
$524{,}288 = 2^{19}$	2	$\log_2 524{,}288 = 19$	$\lfloor \log_2 524{,}288 \rfloor + 1 = 20$
$1{,}048{,}576 = 2^{20}$	2	$\log_2 1{,}048{,}576 = 20$	$\lfloor \log_2 1{,}048{,}576 \rfloor + 1 = 21$

Summary

Key points on natural numbers:

- The sequence $(d_{k-1} \dots d_1 d_0)_b$ represents the natural number $\sum_{i=0}^{k-1} d_i \, b^i$
- With k digits in base b we represent exactly the b^k natural numbers of the interval $[0, b^k - 1]$
- A natural number $n \geq 1$ needs at least $\lfloor \log_b n \rfloor + 1$ digits in base b

Representation of integer numbers

Introduction: Representation of integer numbers

- Integer numbers include both positive and negative values, plus zero
- With k bits we still have only 2^k different sequences, but now we have to use them to represent the **sign** as well
- Two approaches:
  - **sign-magnitude**, the most natural
  - **two's complement**, the standard in modern computers because of its arithmetic advantages
- In this section the base is always $b = 2$

Sign-magnitude representation

Definition 4: sign-magnitude representation

*In the **sign-magnitude representation**, an integer number n is represented using exactly $k \geq 2$ bits, where:*

- *the most significant bit represents the **sign**: 0 if $n \geq 0$ and 1 if $n < 0$;*
- *the remaining $k - 1$ bits represent the **magnitude** $|n|$ (absolute value) in base 2.*

Observation 3

Given k bits, the ***largest integer number*** *that can be represented in sign-magnitude is:*

$$2^{k-1} - 1$$

and the associated ***representation interval*** *for the integer numbers is:*

$$\left[-(2^{k-1} - 1),\ 2^{k-1} - 1 \right]$$

Proof

The magnitude is a natural number written with $k-1$ bits.

By Observation 1 with $b=2$ and $k-1$ digits, the largest magnitude consists of the digit 1 repeated $k-1$ times:

$$\sum_{i=0}^{k-2} 1 \cdot 2^i = \frac{2^{k-1}-1}{2-1} = 2^{k-1}-1$$

- With sign bit 0 we obtain the largest integer $2^{k-1}-1$
- With sign bit 1 we obtain the smallest integer $-(2^{k-1}-1)$
- Every integer in between is represented, since its absolute value is at most $2^{k-1}-1$

□

Example: sign-magnitude with 4 bits (1/2)

With $k = 4$ bits, the first bit is the sign and the last three bits are the magnitude:

Binary	k	Sign	Decimal
$(0000)_2$	4	0 (non-negative)	$+(0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = +0$
$(0001)_2$	4	0 (non-negative)	$+(0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = 1$
$(0010)_2$	4	0 (non-negative)	$+(0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = 2$
$(0011)_2$	4	0 (non-negative)	$+(0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = 3$
$(0100)_2$	4	0 (non-negative)	$+(1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = 4$
$(0101)_2$	4	0 (non-negative)	$+(1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = 5$
$(0110)_2$	4	0 (non-negative)	$+(1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = 6$
$(0111)_2$	4	0 (non-negative)	$+(1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = 7$

Example: sign-magnitude with 4 bits (2/2)

Binary	k	Sign	Decimal
$(1000)_2$	4	1 (negative)	$-(0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = -0$
$(1001)_2$	4	1 (negative)	$-(0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = -1$
$(1010)_2$	4	1 (negative)	$-(0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = -2$
$(1011)_2$	4	1 (negative)	$-(0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = -3$
$(1100)_2$	4	1 (negative)	$-(1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) = -4$
$(1101)_2$	4	1 (negative)	$-(1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) = -5$
$(1110)_2$	4	1 (negative)	$-(1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = -6$
$(1111)_2$	4	1 (negative)	$-(1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) = -7$

The representation interval is $[-7, 7]$, as stated by Observation 3 with $2^{4-1} - 1 = 7$.

Weak points of sign-magnitude (1/2)

The table of the example shows the two weak points of sign-magnitude.

1. **Zero has two representations**
   - $(0000)_2 = +0$ and $(1000)_2 = -0$
   - Hence only $2^k - 1$ different integers are represented with 2^k sequences

Weak points of sign-magnitude (2/2)

2. **Addition is not the addition of natural numbers**
   - The addition of two integers cannot be performed as the addition of the two sequences seen as natural numbers
   - For example, $3 + (-1) = 2$, but adding $(0011)_2$ and $(1001)_2$ as natural numbers gives

     $$3 + 9 = 12 = (1100)_2$$

     which in sign-magnitude represents -4
   - The sign bits must be handled separately, and this complicates arithmetic operations

For these reasons sign-magnitude is not the standard representation of integers in computers.

Two's complement representation

Two's complement solves both problems:

zero has a unique representation,

and addition works exactly as for natural numbers.

Definition 5: two's complement representation

*In the **two's complement representation** (base 2), an integer n is represented using exactly k bits as follows:*

- *if $n \geq 0$, use the standard binary representation of n with k bits;*
- *if $n < 0$, the representation of n is the binary representation of the natural number $2^k + n$ with k bits.*

The sign bit in two's complement

In two's complement, the most significant bit indicates the sign:

- 0: non-negative number;
- 1: negative number.

To read a sequence whose most significant bit is 1, compute its value r as a natural number: the integer represented is $r - 2^k$.

Properties of two's complement

- Zero has a unique representation: $(00\ldots0)_2$
- Addition and subtraction are performed in the same way as for natural numbers, modulo 2^k
- This is the standard representation used in modern computers

Example: two's complement with 4 bits (1/2)

With $k = 4$ bits, the sequences with most significant bit 0 represent $0, 1, \ldots, 7$:

Binary	k	Sign	Decimal
$(0000)_2$	4	non-negative	0
$(0001)_2$	4	non-negative	1
$(0010)_2$	4	non-negative	2
$(0011)_2$	4	non-negative	3
$(0100)_2$	4	non-negative	4
$(0101)_2$	4	non-negative	5
$(0110)_2$	4	non-negative	6
$(0111)_2$	4	non-negative	7

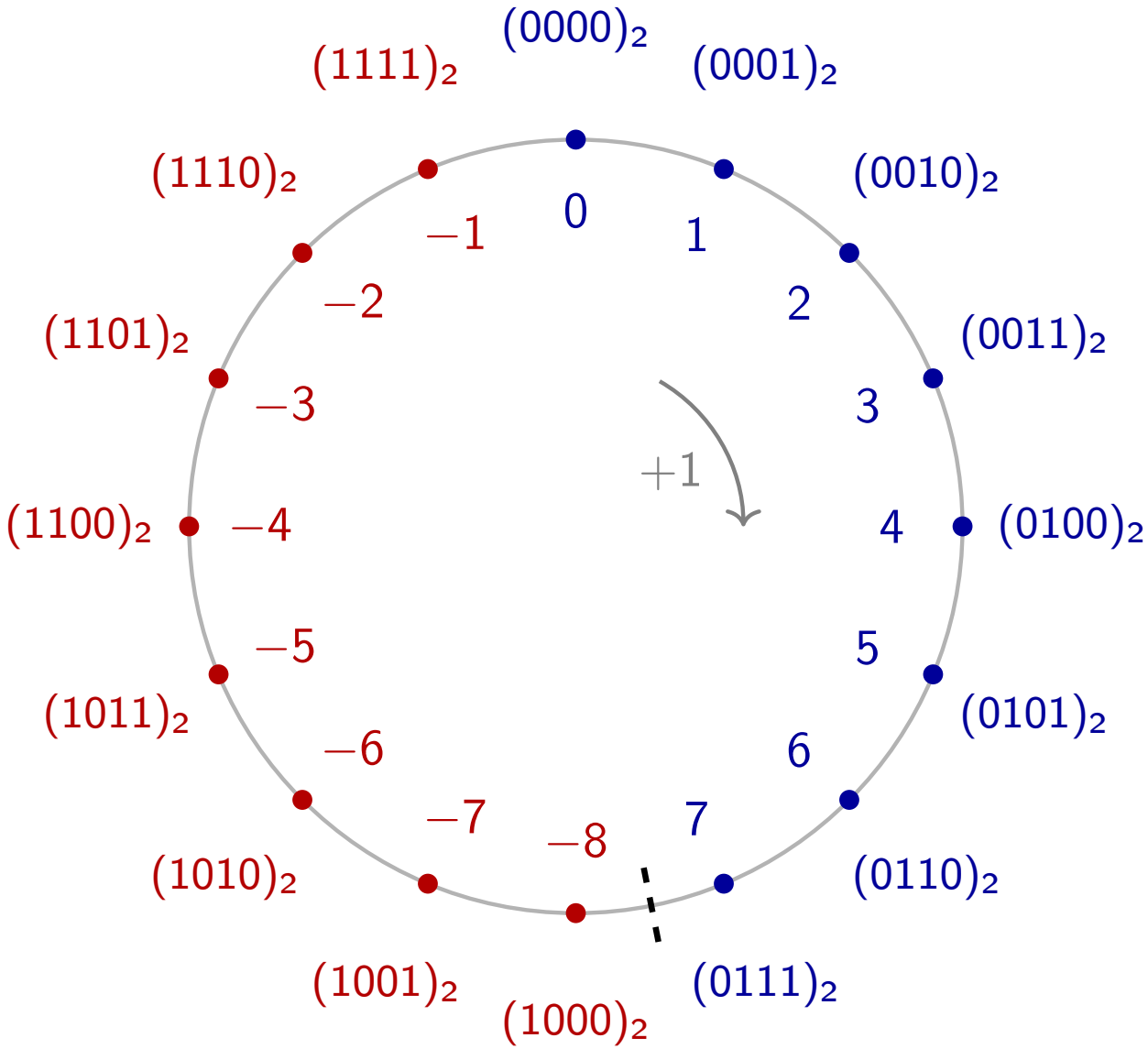
Example: two's complement with 4 bits (2/2)

The sequences with most significant bit 1 represent their natural value minus $2^4 = 16$:

Binary	k	Sign	Decimal
$(1000)_2$	4	negative	$8 - 16 = -8$
$(1001)_2$	4	negative	$9 - 16 = -7$
$(1010)_2$	4	negative	$10 - 16 = -6$
$(1011)_2$	4	negative	$11 - 16 = -5$
$(1100)_2$	4	negative	$12 - 16 = -4$
$(1101)_2$	4	negative	$13 - 16 = -3$
$(1110)_2$	4	negative	$14 - 16 = -2$
$(1111)_2$	4	negative	$15 - 16 = -1$

The two's complement wheel

- The next figure arranges the 16 sequences of the example on a circle
- Moving one step clockwise adds 1 to the natural value of the sequence, modulo $2^4 = 16$
- Non-negative integers in blue (most significant bit 0), negative integers in red (most significant bit 1)
- The integers represented grow from -8 to 7: the dashed mark between $(0111)_2 = 7$ and $(1000)_2 = -8$ is the only point where the integer represented jumps


$(0000)_2$
$(0001)_2$
$(0010)_2$
$(0011)_2$
$(0100)_2$
$(0101)_2$
$(0110)_2$
$(0111)_2$
$(1000)_2$
$(1001)_2$
$(1010)_2$
$(1011)_2$
$(1100)_2$
$(1101)_2$
$(1110)_2$
$(1111)_2$
0
1
2
3
4
5
6
7
−8
−7
−6
−5
−4
−3
−2
−1
+1

Computing $2^k + n$ by a subtraction is not the only way to obtain the representation of a negative integer.

The following observation gives a procedure that works directly on the bits.

Observation 4

Let n be an integer with $-2^{k-1} \leq n < 0$. The two's complement representation of n with k bits is obtained from the binary representation of $|n|$ with k bits in two steps:

1. *invert all the bits (each 0 becomes 1 and each 1 becomes 0);*
2. *add 1 to the result, as a natural number.*

Proof

Let $(d_{k-1} \dots d_1 d_0)_2$ be the binary representation of $|n|$ with k bits.

Inverting the bits gives the digits $1 - d_i$, hence the natural number:

$$\sum_{i=0}^{k-1} (1 - d_i)\, 2^i = \sum_{i=0}^{k-1} 2^i - \sum_{i=0}^{k-1} d_i\, 2^i$$

$$= (2^k - 1) - |n|$$

- Adding 1 we obtain $2^k - |n| = 2^k + n$, which is the two's complement representation of n by Definition 5
- Since $1 \leq |n| \leq 2^{k-1}$, we have $2^{k-1} \leq 2^k + n \leq 2^k - 1$, so the result still has k bits

□

Example: two's complement of -5 with 8 bits

We compute the representation of $n = -5$ with $k = 8$ bits in both ways.

- Step 1: binary representation of $|n| = 5$ with 8 bits: $(00000101)_2$
- Step 2: invert all the bits: $(11111010)_2$, whose natural value is $250 = (2^8 - 1) - 5$
- Step 3: add 1: $(11111010)_2 + 1 = (11111011)_2$
- Check with Definition 5:

$$\begin{aligned} 2^8 + (-5) &= 256 - 5 = 251 \\ &= 128 + 64 + 32 + 16 + 8 + 2 + 1 = (11111011)_2 \end{aligned}$$

Representation interval

Which integers can be represented
with k bits in two's complement?

Observation 5

Given k bits, the largest integer that can be represented in two's complement is:

$$2^{k-1} - 1$$

the smallest integer that can be represented is:

$$-2^{k-1}$$

and therefore the representation interval is:

$$\left[-2^{k-1}, 2^{k-1} - 1 \right]$$

Proof

With k bits, there are 2^k possible binary strings, whose natural values are $0, 1, \ldots, 2^k - 1$.

- The strings whose most significant bit is 0 have natural values $0, 1, \ldots, 2^{k-1} - 1$ and represent the non-negative integers:

$$0, 1, 2, \ldots, 2^{k-1} - 1$$

- The strings whose most significant bit is 1 have natural values $r = 2^{k-1}, \ldots, 2^k - 1$ and represent the integers $r - 2^k$, i.e., the negative integers:

$$-2^{k-1}, -2^{k-1} + 1, \ldots, -1$$

Hence the largest representable integer is $2^{k-1} - 1$, the smallest representable integer is -2^{k-1}, and the interval is $[-2^{k-1}, 2^{k-1} - 1]$. □

Comparison with sign-magnitude

- Compared with sign-magnitude, the interval contains one more integer, -2^{k-1}
- The sequence $(10\ldots0)_2$, which was -0 in sign-magnitude, now represents -2^{k-1}
- All the 2^k sequences represent different integers

Example: representation intervals in two's complement

k	-2^{k-1}	$2^{k-1}-1$	representation interval
1	-1	0	$[-1, 0]$
2	-2	1	$[-2, 1]$
3	-4	3	$[-4, 3]$
4	-8	7	$[-8, 7]$
8	-128	127	$[-128, 127]$
16	$-32{,}768$	32,767	$[-32{,}768,\ 32{,}767]$
32	$-2{,}147{,}483{,}648$	2,147,483,647	$[-2{,}147{,}483{,}648,\ 2{,}147{,}483{,}647]$[1]
64	-2^{63}	$2^{63}-1$	$[-2^{63},\ 2^{63}-1]$[2]

[1] Representation interval of `int` in C and C++ (4 bytes).
[2] Representation interval of `long long int` in C and C++ (8 bytes); $2^{63} = 9{,}223{,}372{,}036{,}854{,}775{,}808$.

Minimum number of bits

As for the natural numbers, we now read the interval in the opposite direction:

how many bits do we need
to represent a given integer n?

Observation 6

The minimum number of bits needed to represent an integer n in two's complement is the smallest integer $k \geq 1$ such that

$$-2^{k-1} \leq n \leq 2^{k-1} - 1.$$

Proof

By Observation 5, an integer represented with k bits in two's complement belongs to the interval

$$[-2^{k-1}, 2^{k-1} - 1],$$

and every integer of this interval can be represented with k bits.

Therefore, the minimum number of bits required to represent n is exactly the smallest k for which

$$-2^{k-1} \leq n \leq 2^{k-1} - 1.$$

□

As for the natural numbers, this condition can be turned into an explicit formula with logarithms.

Corollary 1

The minimum number of bits needed to represent an integer n in two's complement is:

$$k = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = -1 \\ \lfloor \log_2 n \rfloor + 2 & \text{if } n > 0 \\ \lceil \log_2 |n| \rceil + 1 & \text{if } n < 0 \end{cases}$$

Proof

With $k = 1$ the interval is $[-1, 0]$, which gives the first case.

- If $n > 0$, the condition of Observation 6 reduces to

$$n \leq 2^{k-1} - 1 \iff n < 2^{k-1} \iff \log_2 n < k - 1$$

The smallest integer $k - 1$ strictly greater than $\log_2 n$ is $\lfloor \log_2 n \rfloor + 1$, hence $k = \lfloor \log_2 n \rfloor + 2$.

- If $n < 0$, the condition reduces to

$$-2^{k-1} \leq n \iff |n| \leq 2^{k-1} \iff \log_2 |n| \leq k - 1$$

The smallest integer $k - 1$ greater than or equal to $\log_2 |n|$ is $\lceil \log_2 |n| \rceil$, hence $k = \lceil \log_2 |n| \rceil + 1$ (for $n = -1$ this gives $k = 1$, in agreement with the first case).

□

Example: minimum number of bits in two's complement

The last column shows the representation of n with the minimum number of bits.

n	minimum number of bits	representation
-1	1	1
0	1	0
1	2	01
-2	2	10
2	3	010
-4	3	100
4	4	0100
-8	4	1000
8	5	01000
-16	5	10000

For instance, for $n = 8$ Corollary 1 gives $\lfloor \log_2 8 \rfloor + 2 = 3 + 2 = 5$, and for $n = -16$ it gives $\lceil \log_2 16 \rceil + 1 = 4 + 1 = 5$.

Addition and overflow

- The main advantage of two's complement: integers are added **exactly as natural numbers**
- We add the two sequences of k bits column by column, from right to left, with carries
- We discard the carry that goes out of the most significant position

Observation 7

Let n_1 and n_2 be two integers represented with k bits in two's complement. Add their representations as natural numbers and keep only the k least significant bits of the sum.

- *If $n_1 + n_2 \in [-2^{k-1}, 2^{k-1} - 1]$, the result is the two's complement representation of $n_1 + n_2$.*
- *Otherwise, the sum cannot be represented with k bits and we have an **overflow**. An overflow occurs if and only if n_1 and n_2 have the same sign and the result has the opposite sign bit.*

Proof (1/3)

Let r_1 and r_2 be the natural values of the two representations.

- By Definition 5, $r_j = n_j$ or $r_j = 2^k + n_j$
- So $r_1 + r_2$ differs from $n_1 + n_2$ by 0, 2^k or 2^{k+1}
- Keeping the k least significant bits means subtracting 2^k when $r_1 + r_2 \geq 2^k$ (the discarded carry)

Hence the result r satisfies

$$r \in [0, 2^k - 1] \quad \text{and} \quad r - (n_1 + n_2) \text{ is a multiple of } 2^k$$

Proof (2/3)

The sum is in the interval.

If $n_1 + n_2 \in [-2^{k-1}, 2^{k-1} - 1]$:

- its representation $\bar{r}$ also lies in $[0, 2^k - 1]$
- $\bar{r} - (n_1 + n_2)$ is 0 or 2^k
- then $r - \bar{r}$ is a multiple of 2^k with $|r - \bar{r}| \leq 2^k - 1$, so $r = \bar{r}$

Different signs.

If n_1 and n_2 have different signs, $n_1 + n_2$ lies between n_1 and n_2, which belong to the interval: no overflow.

Proof (3/3)

Both non-negative.

- $n_1 + n_2 \in [0, 2^k - 2]$, no carry is discarded and $r = n_1 + n_2$
- An overflow occurs if and only if $n_1 + n_2 \geq 2^{k-1}$, i.e., if and only if the sign bit of r is 1

Both negative.

- $r_1 + r_2 = 2^{k+1} + n_1 + n_2 \geq 2^k$, so $r = 2^k + n_1 + n_2$
- An overflow occurs if and only if $n_1 + n_2 < -2^{k-1}$, i.e., if and only if $r < 2^{k-1}$, i.e., the sign bit of r is 0

□

How to read the column additions

- In the examples, the first row of each addition contains the **carries**
- The carry written above a column is the one coming from the column on its right
- The leftmost column receives the carry out of the most significant position, which is discarded

Example: addition without overflow

We compute $3 + (-1)$ with $k = 4$ bits, interval $[-8, 7]$.

- Step 1: representations: $3 = (0011)_2$ and $-1 \to 2^4 - 1 = 15 = (1111)_2$
- Step 2: column addition:

carry	1	1	1	1	
		0	0	1	1
		1	1	1	1
sum	1	0	0	1	0

- Step 3: discard the carry out: the result is $(0010)_2 = 2 = 3 + (-1)$

Example: addition with overflow

We compute $6 + 3$ with $k = 4$ bits, interval $[-8, 7]$. Since $6 + 3 = 9 > 7$, an overflow is expected.

- Step 1: representations: $6 = (0110)_2$ and $3 = (0011)_2$
- Step 2: column addition:

carry	0	1	1	0	
		0	1	1	0
		0	0	1	1
sum	0	1	0	0	1

- Step 3: the result $(1001)_2$ has sign bit 1 and represents $9 - 16 = -7$, although both operands are positive: an overflow has occurred

Summary

Key points on integer numbers:

- Sign-magnitude with k bits: interval $[-(2^{k-1}-1), 2^{k-1}-1]$, two representations of zero, addition requires a special treatment of the sign
- Two's complement with k bits: a negative n is represented by $2^k + n$ (equivalently: invert the bits of $|n|$ and add 1); interval $[-2^{k-1}, 2^{k-1}-1]$; a unique zero
- In two's complement, addition is the addition of natural numbers modulo 2^k; an overflow occurs only when two operands with the same sign give a result with the opposite sign

Representation of real numbers

Introduction: Representation of real numbers

- Natural and integer numbers are represented exactly, as long as they lie in the representation interval
- Real numbers are different: between any two real numbers there are infinitely many others
- Most real numbers (for instance $\frac{1}{3}$ or $\sqrt{2}$) have infinitely many digits in any base
- With a finite number of bits, a real number can be represented, in general, only **approximately**
- We study the floating-point representation used by computers

Digits after the point

- Positional numeral systems extend to the digits after the point
- The digits on the right of the point are multiplied by the negative powers of the base

Definition 6: sequence of digits with a fractional part

A sequence of k digits before the point and h digits after the point in base b is denoted as:

$$\left(d_{k-1} \ldots d_1 \, d_0 \, . \, d_{-1} \, d_{-2} \ldots d_{-h}\right)_b$$

and represents the number:

$$\sum_{i=-h}^{k-1} d_i \; b^i$$

Example: binary numbers with a fractional part

- $(101.11)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2}$

$$= 4 + 1 + 0.5 + 0.25 = 5.75$$

- $(0.011)_2 = 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.25 + 0.125 = 0.375$
- The decimal number 0.1 has no finite binary representation: its binary digits after the point repeat forever,

$$0.1 = (0.0\,0011\,0011\,0011\ldots)_2$$

Fixed point and floating point

- Writing a number with a fixed number of digits before and after the point (**fixed point**) wastes digits:
  - very large numbers need many digits before the point
  - very small numbers need many digits after it

- The **floating-point** representation lets the point "float", as in the scientific notation

$$6.02 \times 10^{23}$$

Definition 7: normalized binary scientific notation

*Every real number $x \neq 0$ can be written in the **normalized binary scientific notation**:*

$$x = (-1)^s \cdot (1.f_1 f_2 f_3 \dots)_2 \cdot 2^e$$

*where $s \in \{0, 1\}$ is the **sign**, the sequence $(1.f_1 f_2 f_3 \dots)_2$ is the **significand** (or **mantissa**), whose bits $f_1, f_2, f_3, \dots$ after the point form the **fraction**, and the integer e is the **exponent**.*

The significand

- The significand is always between 1 (included) and 2 (excluded)
- Its first digit is always 1, so it does not need to be stored
- For example:

$$5.75 = (101.11)_2 = (1.0111)_2 \cdot 2^2$$

the point moves two positions to the left and the exponent is 2

The IEEE 754 standard

- Computers store floating-point numbers according to the **IEEE 754** standard
  - IEEE: Institute of Electrical and Electronics Engineers
- We describe its two most common formats, focusing on the 32-bit one

Definition 8: IEEE 754 single precision

*In the **IEEE 754 single precision** format, a real number is stored with 32 bits divided into three fields:*

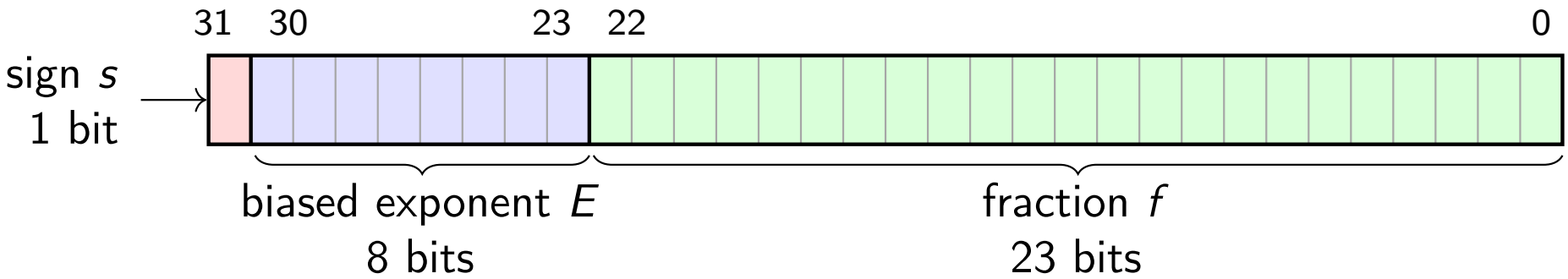
- *the **sign** s: 1 bit;*
- *the **biased exponent** E: 8 bits, storing the natural number $E = e + 127$, where e is the exponent and 127 is the **bias**;*
- *the **fraction** f: 23 bits, storing the bits $f_1 f_2 \dots f_{23}$ of the significand after the point.*

*For $1 \leq E \leq 254$ (**normalized numbers**), the number represented is:*

$$x = (-1)^s \cdot (1.f_1 f_2 \dots f_{23})_2 \cdot 2^{E-127}$$

The 32 bits of single precision

The 32 bits of the IEEE 754 single precision format, numbered from 31 (most significant) to 0 (least significant):



The exponent field

- The exponent is stored with a bias instead of in two's complement, so that E is a natural number between 0 and 255
- The two extreme values of E are reserved:
  - $E = 0$ is used for zero (all fraction bits equal to 0; there are two zeros, $+0$ and -0, as in sign-magnitude) and for very small numbers called *subnormal*
  - $E = 255$ is used for $+\infty$ and $-\infty$ (fraction equal to 0, e.g., the result of $1/0$) and for *NaN*, "Not a Number" (fraction different from 0, e.g., the result of $0/0$)

Single and double precision

- The **double precision** format has the same structure with 64 bits: 1 bit of sign, 11 bits of exponent with bias 1023, and 52 bits of fraction

Format	total bits	sign	exponent	fraction	bias
single precision (C++ `float`)	32	1	8	23	127
double precision (C++ `double`)	64	1	11	52	1023

Example: from a real number to its IEEE 754 single precision representation

We represent $x = 5.75$ in single precision.

- Step 1 (binary): $5.75 = 4 + 1 + 0.5 + 0.25 = (101.11)_2$
- Step 2 (normalization): $(101.11)_2 = (1.0111)_2 \cdot 2^2$, hence $s = 0$, $e = 2$ and the fraction starts with 0111
- Step 3 (biased exponent): $E = e + 127 = 129 = 128 + 1 = (10000001)_2$
- Step 4 (fraction): the bits after the point, completed with zeros up to 23 bits: 01110000000000000000000

The 32 bits are:

$$\underbrace{0}_{s} \quad \underbrace{10000001}_{E} \quad \underbrace{01110000000000000000000}_{f}$$

Example: from an IEEE 754 single precision representation to a real number

We read the 32 bits:

$$\underbrace{1}_{s} \quad \underbrace{10000010}_{E} \quad \underbrace{01000000000000000000000}_{f}$$

- Step 1 (sign): $s = 1$, the number is negative
- Step 2 (exponent): $E = (10000010)_2 = 128 + 2 = 130$, hence $e = E - 127 = 3$
- Step 3 (significand): $(1.01)_2 = 1 + 0.25 = 1.25$
- Step 4 (value): $x = (-1)^1 \cdot 1.25 \cdot 2^3 = -10$

Rounding

When the binary digits of a number
do not fit in the 23 bits of the fraction,

the number is **rounded**
to the nearest representable one.

Example: the number 0.1 is not representable exactly (1/2)

- From the example on binary numbers with a fractional part:

$$0.1 = (0.0\,0011\,0011\ldots)_2 = (1.1001\,1001\,1001\ldots)_2 \cdot 2^{-4}$$

  whose fraction never ends

- In single precision the fraction is rounded to 23 bits, and the bits stored are:

$$\underbrace{0}_{s} \quad \underbrace{01111011}_{E=123} \quad \underbrace{10011001100110011001101}_{f}$$

Example: the number 0.1 is not representable exactly (2/2)

- The number actually stored is 0.1000000014901161193847656 25, not 0.1
- In double precision the error is smaller but still present: the stored value is

$$0.1000000000000000055511151231\ldots$$

- For this reason the computation $0.1 + 0.2$ in double precision gives

$$0.30000000000000004$$

and not the double precision value of 0.3

Range and precision

- The finite number of bits of the **exponent** limits the **range** of the representable numbers
- The finite number of bits of the **fraction** limits their **precision**

Observation 8

In IEEE 754 single precision:

- *the largest finite number is* $(2 - 2^{-23}) \cdot 2^{127} \approx 3.4028235 \times 10^{38}$*;*
- *the smallest positive normalized number is* $2^{-126} \approx 1.1754944 \times 10^{-38}$*;*
- *the distance between* 1 *and the next representable number is* $2^{-23} \approx 1.1920929 \times 10^{-7}$ *(called the* ***machine epsilon****).*

Proof (1/2)

Largest finite number.

It has $s = 0$, the largest normalized biased exponent $E = 254$, i.e., $e = 127$, and all 23 fraction bits equal to 1. Its significand is:

$$(1.\underbrace{11\ldots1}_{23})_2 = \sum_{i=0}^{23} 2^{-i} = 2 - 2^{-23}$$

hence the number is $(2 - 2^{-23}) \cdot 2^{127}$.

Proof (2/2)

Smallest positive normalized number.

It has $E = 1$, i.e., $e = -126$, and all fraction bits equal to 0, hence it is $1 \cdot 2^{-126}$.

Machine epsilon.

We have $1 = (1.00\ldots0)_2 \cdot 2^0$. The next representable number is obtained by setting to 1 the last fraction bit, which has weight 2^{-23} when $e = 0$: it is $1 + 2^{-23}$. □

Relative precision

- The machine epsilon measures the **relative precision** of the format:
  - single precision corresponds to about 7 significant decimal digits
  - double precision (machine epsilon $2^{-52} \approx 2.2 \times 10^{-16}$) to about 16
- As a consequence, two expressions that are mathematically equal may give different floating-point results
- Floating-point numbers should not be compared with the equality operator without care

Summary

Key points on real numbers:

- A floating-point number is stored as a sign, an exponent and a fraction: $x = (-1)^s \cdot (1.f)_2 \cdot 2^{E-\text{bias}}$
- IEEE 754 single precision: $1 + 8 + 23$ bits, bias 127; double precision: $1 + 11 + 52$ bits, bias 1023
- The exponent bits determine the range, the fraction bits the precision; most real numbers (for instance 0.1) are stored with a rounding error

In summary

Introduction: In summary

- We collect the three representations of the chapter
- For each of them: what can be represented with a given number of digits, and the main properties

Summary (1/2)

Natural numbers (k digits in base b):

- value $\sum_{i=0}^{k-1} d_i \, b^i$, interval $[0, b^k - 1]$
- at least $\lfloor \log_b n \rfloor + 1$ digits for $n \geq 1$

Integer numbers (k bits):

- sign-magnitude: interval $[-(2^{k-1} - 1), 2^{k-1} - 1]$, two zeros
- two's complement: $n < 0$ stored as $2^k + n$, interval $[-2^{k-1}, 2^{k-1} - 1]$, one zero
- two's complement addition: natural addition modulo 2^k, overflow only for operands with the same sign

Summary (2/2)

Real numbers (floating point):

- $x = (-1)^s \cdot (1.f)_2 \cdot 2^{E-\text{bias}}$
- IEEE 754: single precision $1 + 8 + 23$ bits, bias 127; double precision $1 + 11 + 52$ bits, bias 1023
- the exponent gives the range, the fraction the precision
- most real numbers, such as 0.1, are stored with a rounding error